

Algoritmos de Búsqueda II

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo



Algoritmos de Búsqueda II

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

Objetivo General

Analizar e implementar algoritmos de búsqueda en estructuras lineales como arreglos y vectores, identificando sus características, condiciones de uso y eficiencia, tanto en estructuras ordenadas como no ordenadas

Objetivos Específicos

- Explicar el funcionamiento y diferencias entre la búsqueda lineal y la búsqueda binaria, destacando sus requisitos y aplicaciones más comunes.
- Aplicar ambos algoritmos en estructuras de datos unidimensionales, evaluando su comportamiento y rendimiento según el tipo y orden de los datos.

Búsqueda binaria

Consiste en **examinar primero el elemento que está en el centro del arreglo**. Si el valor buscado es menor, se continúa la búsqueda en la mitad inferior del arreglo usando el mismo método; si es mayor, se busca en la mitad superior. **La búsqueda binaria necesita menos pasos y comparaciones** que la búsqueda lineal, pero solo funciona si el arreglo está previamente ordenado.

Ventajas

- **Es mucho más rápida que la búsqueda secuencial**, especialmente en arreglos grandes. Tiene una complejidad logarítmica

Desventajas

- **Esta búsqueda solo funciona si** los elementos están ordenados de menor a mayor. Si no lo están, primero hay que ordenarlos, y eso puede tardar más.

Ejemplos de búsqueda binaria

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      // Declaramos un arreglo ordenado de forma ascendente
6      int arreglo[] = {3, 8, 15, 23, 30, 42, 56, 70, 81};
7      int tamano = sizeof(arreglo) / sizeof(arreglo[0]);
8
9      int valor; // Valor que el usuario desea buscar
10     bool encontrado = false;
11
12     cout << "Ingrese el valor que desea buscar: ";
13     cin >> valor;
14
15     // Variables para manejar los extremos del arreglo
16     int inicio = 0;
17     int fin = tamano - 1;
18
19     while (inicio <= fin) {
20         int medio = (inicio + fin) / 2; // Calculamos la posición central
21
22         if (arreglo[medio] == valor) {
23             cout << "✅ Valor encontrado en la posición " << medio << endl;
24             encontrado = true;
25             break;
26         } else if (valor < arreglo[medio]) {
27             fin = medio - 1; // Buscar en la mitad izquierda
28         } else {
29             inicio = medio + 1; // Buscar en la mitad derecha
30         }
31     }
32
33     if (!encontrado) {
34         cout << "❌ Valor no encontrado en el arreglo." << endl;
35     }
36
37     return 0;
38 }
```

■ Explicación del código

1. Se declara un **arreglo ordenado** (condición obligatoria para la búsqueda binaria).
2. El usuario escribe el número que quiere buscar.
3. Se usan dos variables: inicio y fin, para definir el rango de búsqueda.
4. En cada paso, se calcula el medio y se compara con el valor buscado:
 - Si lo encuentra → lo muestra y termina.
 - Si es menor → busca en la mitad izquierda.
 - Si es mayor → busca en la mitad derecha.
5. Si no lo encuentra, informa al usuario.

Aunque la búsqueda binaria puede implementarse con un ciclo `for`, en la práctica se prefiere el uso de `while`, ya que permite controlar con claridad el rango variable de búsqueda (`inicio` y `fin`) mientras se divide el arreglo.
`while` es más flexible cuando no se conoce cuántas iteraciones serán necesarias.

Output

```
Ingrese el valor que desea buscar: 23
✅ Valor encontrado en la posición 3
```

```
for (int inicio = 0, fin = tamano - 1; inicio <= fin; ) {
    int medio = (inicio + fin) / 2;
    if (arreglo[medio] == valor) { /* encontrado */ }
    else if (valor < arreglo[medio]) fin = medio - 1;
    else inicio = medio + 1;
}
```

Nota: Recuerda que puedes probar el código con un compilador en línea.
<https://www.programiz.com/cpp-programming/online-compiler>

Ejemplos de búsqueda binaria con ordenamiento

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      // Arreglo desordenado
6      int arreglo[] = {42, 3, 81, 15, 8, 56, 30, 70, 23};
7      int tamano = sizeof(arreglo) / sizeof(arreglo[0]);
8
9      int valor;
10     bool encontrado = false;
11
12     // Ordenamos el arreglo con el método burbuja
13     for (int i = 0; i < tamano - 1; i++) {
14         for (int j = 0; j < tamano - i - 1; j++) {
15             if (arreglo[j] > arreglo[j + 1]) {
16                 // Intercambiar los elementos
17                 int temp = arreglo[j];
18                 arreglo[j] = arreglo[j + 1];
19                 arreglo[j + 1] = temp;
20             }
21         }
22     }
23
24     // Mostrar el arreglo ya ordenado
25     cout << "Arreglo ordenado: ";
26     for (int i = 0; i < tamano; i++) {
27         cout << arreglo[i] << " ";
28     }
29     cout << endl;
30
31     // Solicitar el valor a buscar
32     cout << "Ingrese el valor que desea buscar: ";
33     cin >> valor;
34 }
```

```
35 // Búsqueda binaria
36 int inicio = 0, fin = tamano - 1;
37 while (inicio <= fin) {
38     int medio = (inicio + fin) / 2;
39
40     if (arreglo[medio] == valor) {
41         cout << "✓ Valor encontrado en la posición " << medio << endl;
42         encontrado = true;
43         break;
44     } else if (valor < arreglo[medio]) {
45         fin = medio - 1;
46     } else {
47         inicio = medio + 1;
48     }
49 }
50
51 if (!encontrado) {
52     cout << "✗ Valor no encontrado en el arreglo." << endl;
53 }
54
55 return 0;
56 }
```

Output

```
Arreglo ordenado: 3 8 15 23 30 42 56 70 81
Ingrese el valor que desea buscar: 42
✓ Valor encontrado en la posición 5
```



¿Cómo funciona el ordenamiento burbuja?

Compara elementos de dos en dos y los va intercambiando de lugar si están desordenados, hasta que todo el arreglo quede en orden ascendente.



¿Cómo funciona paso a paso?

- 1.El ciclo externo (i) controla **cuántas veces se repite** el proceso de revisión del arreglo.
 - Se repite tamaño - 1 veces, ya que en cada pasada se "empuja" el valor más grande hacia el final.
- 2.El ciclo interno (j) compara **pares de elementos consecutivos**: arreglo[j] y arreglo[j + 1].
- 3.Si el valor actual (arreglo[j]) es **mayor** que el siguiente (arreglo[j + 1]), se **intercambian**.
- 4.Este proceso hace que el número más grande "suba" al final del arreglo en cada pasada, como una **burbuja** que flota hasta la superficie

Nota: Recuerda que puedes probar el código con un compilador en línea.
<https://www.programiz.com/cpp-programming/online-compiler>

Algoritmos de Búsqueda II

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

Algoritmos de Búsqueda II

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

